

Implementación y Análisis de Complejidad de Listas, Pilas y Colas en Java

Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial
Estructura de Datos (2026-2)

Repositorio GitHub: https://github.com/jpiratobat/ED_Listas_Pilas_Colas.git

Jorge Eduardo Piratoba Tocarruncho

jpiratobat@unal.edu.co

1. Objetivo

El presente informe tiene como objetivo abordar el diseño, la implementación y el análisis de complejidad de las estructuras de datos lineales fundamentales: **Listas Enlazadas** (*Singly Linked-List*), **Listas Doblemente Enlazadas** (*Doubly LinkedList*), **Pilas** (*DynamicArrayStack*) y **Colas** (*DynamicArrayQueue*). Estas estructuras conforman la base teórica y práctica para el desarrollo de sistemas de software eficiente, optimización del uso de memoria y ejecución de algoritmos complejos.

En este laboratorio se analizan las operaciones de inserción, eliminación y búsqueda, evaluando su comportamiento dinámico mediante el uso de anotaciones Big- O .

2. Implementación

2.1. Interfaz Lista (*MyList*)

Define la estructura contractual e interfaz genérica que establece las operaciones fundamentales para la gestión de datos lineales de tipo secuencial.

- `pushFront(T data)`: Inserta un nuevo elemento al principio de la lista.
- `pushBack(T data)`: Inserta un nuevo elemento al final de la lista.
- `topFront()`: Devuelve el primer elemento de la lista sin eliminarlo.
- `popFront()`: Extrae y devuelve el primer elemento de la lista.
- `topBack()`: Devuelve el último elemento de la lista sin eliminarlo.
- `popBack()`: Extrae y devuelve el último elemento de la lista.
- `isEmpty()`: Determina si la lista no contiene elementos.
- `getSize()`: Devuelve la cantidad de elementos almacenados en la lista.
- `erase(T data)`: Elimina todas las apariciones del dato indicado en la lista.

2.2. Lista Simplemente Enlazada (*SinglyLinkedList*)

Una lista simplemente enlazada es una estructura de datos lineal formada por una secuencia de nodos conectados de forma unidireccional mediante referencias. A diferencia de los arreglos estáticos, no requiere un bloque de memoria contiguo.

- **Nodo:** Estructura básica que almacena el elemento (*data*) y una referencia (*next*) al siguiente nodo.
- **Cabeza (*Head*):** Puntero de entrada al primer nodo de la lista.
- **Cola (*Tail*):** Puntero directo al último nodo de la lista, optimizando las inserciones al final a $O(1)$.

2.3. Lista Doblemente Enlazada (*DoublyLinkedList*)

A diferencia de la lista simplemente enlazada, los nodos en esta estructura cuentan con punteros bidireccionales. Aunque ofrece mayor flexibilidad de recorrido (hacia adelante y hacia atrás), incrementa el consumo de memoria al almacenar una referencia adicional por nodo.

- **Nodo Doble:** Contiene el dato almacenado y dos referencias: *next* (siguiente nodo) y *prev* (nodo anterior).
- **Cabeza (*Head*):** Puntero al primer nodo de la lista.
- **Cola (*Tail*):** Puntero al último nodo de la lista.

2.4. Pila (*MyStack*)

Estructura de datos de acceso restringido que opera bajo la filosofía **LIFO** (*Last In, First Out*), donde el último elemento en ingresar es el primero en ser extraído.

- **push(T data):** Agrega un elemento a la parte superior (*top*) de la pila.
- **pop():** Elimina y retorna el elemento en la parte superior.
- **peek():** Consulta el elemento en la parte superior sin retirarlo.
- **isEmpty():** Retorna un valor booleano que indica si la pila está vacía.
- **getSize():** Retorna la cantidad total de elementos contenidos en la pila.
- **delete(T data):** Elimina las apariciones del valor especificado.

2.5. Pila sobre Arreglo Dinámico (*DynamicArrayStack*)

Implementación de la pila sobre un arreglo de capacidad variable. Cuando la pila alcanza su capacidad máxima al ejecutar un **push**, el arreglo redimensiona dinámicamente su tamaño, duplicando su capacidad actual. Esto implica la creación de un nuevo arreglo en memoria y la copia secuencial de todos los elementos preexistentes.

2.6. Cola (*MyQueue*)

Estructura de datos basada en el principio **FIFO** (*First In, First Out*), donde los elementos se insertan por un extremo (*rear*) y se extraen por el extremo opuesto (*head*).

- `enqueue(T data)`: Añade un nuevo elemento al final de la cola.
- `dequeue()`: Retira y retorna el primer elemento de la cola.
- `front()`: Consulta el primer elemento de la cola sin eliminarlo.
- `isEmpty()`: Retorna un valor booleano indicando si la cola se encuentra vacía.
- `getSize()`: Retorna el número actual de elementos en la cola.
- `delete(T data)`: Elimina las apariciones del dato especificado dentro de la cola.

2.7. Cola sobre Arreglo Dinámico (*DynamicArrayQueue*)

Esta implementación utiliza un “arreglo circular” para evitar mayor consumo en memoria de elementos tras cada operación `dequeue`. Cuando el arreglo alcanza su capacidad máxima, se asigna un nuevo contenedor de mayor tamaño, se reordenan secuencialmente los elementos partiendo desde el índice *front* hacia el nuevo arreglo y se reajustan los punteros principales.

3. Análisis de Complejidades

En las Tablas [1, 2, 3] se presenta el resumen de complejidad teóricos, en notación **Big(O)**, para cada estructura de datos analizada.

Cuadro 1: Complejidad de métodos en *MyList*

Método	Singly (sin Tail)	Singly (con Tail)	Doubly (sin Tail)	Doubly (con Tail)
<code>pushFront(data)</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
<code>pushBack(data)</code>	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
<code>topFront()</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
<code>popFront()</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
<code>topBack()</code>	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
<code>popBack()</code>	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
<code>find()</code>	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
<code>isEmpty()</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
<code>getSize()</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
<code>erase(data)</code>	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
<code>addBefore(target, data)</code>	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
<code>addAfter(target, data)</code>	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$

Cuadro 2: Complejidad de métodos en *MyStack*

Método	Peor Caso	Caso Amortizado
push(data)	$\mathcal{O}(n)$	$\mathcal{O}(1)$
pop()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
peek()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
isEmpty()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
getSize()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
delete(data)	$\mathcal{O}(n)$	$\mathcal{O}(n)$

Cuadro 3: Complejidad de métodos en *MyQueue*

Método	Peor Caso	Caso Amortizado
enqueue(data)	$\mathcal{O}(n)$	$\mathcal{O}(1)$
dequeue()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
front()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
isEmpty()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
getSize()	$\mathcal{O}(1)$	$\mathcal{O}(1)$
delete(data)	$\mathcal{O}(n)$	$\mathcal{O}(n)$

A partir de los datos recolectados en microsegundos (*ms*) mediante el uso de librería, se generaron los gráficos de rendimiento para evaluar el comportamiento temporal según el tamaño de la entrada $N \in \{10, 100, 1000, 10000\}$. Se graficó la información de los Cuadros del 4 al 9, se realizó con la Escala Logarítmica, cuando el tiempo dan 0 microsegundos no es posible graficarlo,

3.1. Análisis de los Resultados en Listas

Se confirma la complejidad con la teoría.

- **Impacto del puntero *Tail*:** En las Figuras 1 y 3, la ausencia de *tail* provoca un crecimiento lineal $\mathcal{O}(n)$ en las operaciones al final (**pushBack** y **topBack**), dado que se debe recorrer la lista nodo por nodo. En contraste, las Figuras 2 y 4 muestran un tiempo constante $\mathcal{O}(1)$ para dichas operaciones.
- **Diferencia en popBack:** Únicamente la estructura *DoublyLinkedListTail* (Figura 4) logra ejecutar popBack en tiempo constante $\mathcal{O}(1)$, debido a que los enlaces hacia atrás (*prev*) permiten desenganchar el último nodo sin recorrer toda la lista.

3.2. Estructuras sobre Arreglos Dinámicos

Las Figuras 6 y 5 representan el rendimiento y costo amortizado de las implementaciones basadas en arreglos redimensionables.

3.3. Análisis del Costo Amortizado

En los gráficos de arreglos dinámicos se aprecian relativamente “constante” en el tiempo de ejecución. Ya que su complejidad correspondiente varía a las operaciones donde el arreglo alcanza su capacidad máxima y se realiza el redimensionamiento y copiado de elementos. Se confirma la complejidad con la teoría.

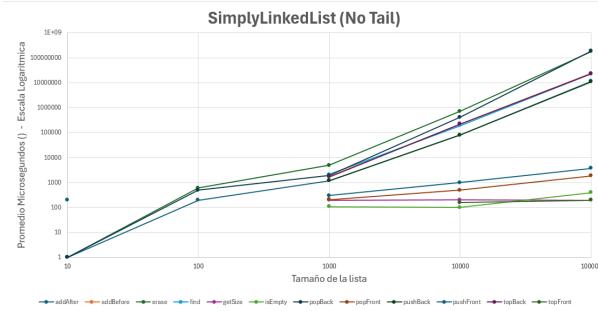


Figura 1: Rendimiento de *SinglyLinkedList* (Sin Tail).

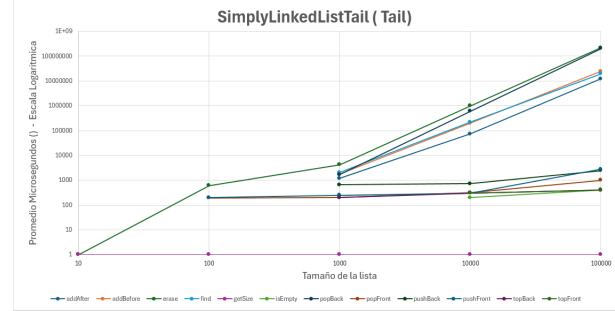


Figura 2: Rendimiento de *SinglyLinkedListTail* (Con Tail).

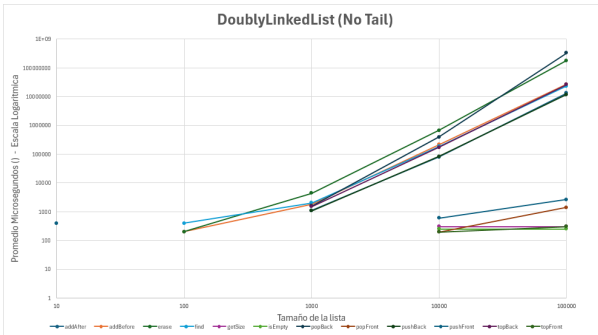


Figura 3: Rendimiento de *DoublyLinkedList* (Sin Tail).

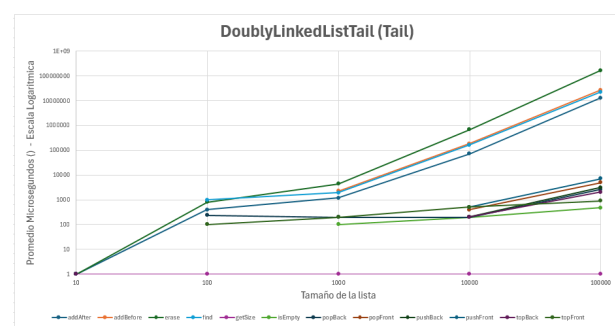


Figura 4: Rendimiento de *DoublyLinkedListTail* (Con Tail).

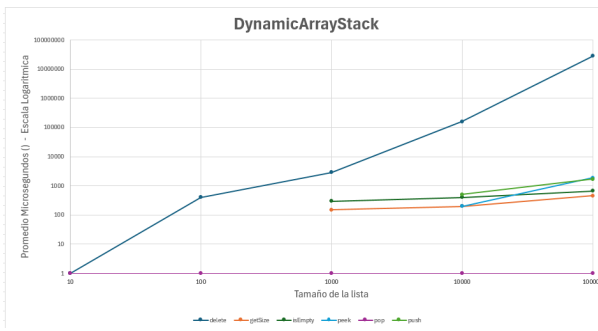


Figura 5: Tiempos de ejecución en *DynamicArrayStack* (push vs pop).

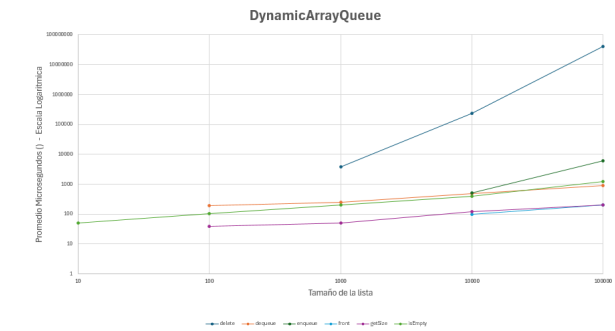


Figura 6: Tiempos de ejecución en *DynamicArrayQueue* (enqueue vs dequeue).

4. Relación entre MyList, MyQueue, MyStack

Las estructuras de datos lineales restringidas como las Pilas (*MyStack*) y las Colas (*MyQueue*) pueden interpretarse como abstracciones especializadas de una Lista (*MyList*). A continuación, se realiza la comparación entre los métodos equivalentes de estas estructuras, con la que cuenta menor complejidad.

1. Inserción en Pila (push) vs. pushFront en Lista:

- **Implementación:** *SinglyLinkedList* (Simple sin Tail).

- **Justificación:** Insertar al inicio de una lista enlazada simple solo requiere actualizar el puntero *head* y la referencia *next* del nuevo nodo en tiempo constante $\mathcal{O}(1)$. No se requiere un puntero *tail* ni dobles enlaces (*prev*), lo que minimiza la sobrecarga de memoria por nodo.

2. Inserción en Cola (enqueue) vs. pushBack en Lista:

- **Implementación :** *SinglyLinkedListTail* (Simple con *Tail*).
- **Justificación:** Al contar con un puntero directo al último nodo (*tail*), la inserción al final se realiza en $\mathcal{O}(1)$ sin necesidad de recorrer la lista completa en $\mathcal{O}(n)$. Mantener una estructura simplemente enlazada es suficiente, evitando el espacio adicional que requeriría una lista doblemente enlazada.

3. Extracción en Pila (pop) y Cola (dequeue) vs. popFront / popBack:

- **Extracción en Pila (pop):** Se equipara con **popFront** en *SinglyLinkedList*, logrando un desenganche directo en $\mathcal{O}(1)$.
- **Extracción en Cola (dequeue):** Se equipara con **popFront** en *SinglyLinkedListTail*, removiendo el elemento del frente en $\mathcal{O}(1)$ mientras se conserva la referencia del final.

4. Eliminación Arbitraria (delete vs. erase):

- **Implementación :** *DoublyLinkedListTail* (Doble con *Tail*).
- **Justificación:** Si se conoce la referencia al nodo, una lista doblemente enlazada permite reenlazar los punteros *prev* y *next* de forma inmediata en $\mathcal{O}(1)$. No obstante, en la búsqueda del valor a eliminar, todas las variantes de lista requieren un recorrido lineal de peor caso $\mathcal{O}(n)$.

5. Conclusiones

5.1. Criterios de Selección y Condiciones de Uso

La elección de la estructura de datos lineal óptima depende directamente del patrón de acceso a los datos y de las restricciones de recursos del sistema:

- ***SinglyLinkedList* (Sin *Tail*):** Es la opción más liviana cuando la aplicación realiza lecturas e inserciones exclusivamente al inicio (**pushFront/popFront**). Su uso es desaconsejado si se requieren inserciones o accesos frecuentes al final de la secuencia, ya que degeneran en $\mathcal{O}(n)$.
- ***SinglyLinkedListTail* (Con *Tail*):** Es la estructura idónea para la implementación de colas basadas en nodos, permitiendo inserciones al final (**pushBack**) y extracciones al inicio (**popFront**) en tiempo estricto $\mathcal{O}(1)$ sin la sobrecarga de un doble enlace.
- ***DoublyLinkedListTail* (Doble con *Tail*):** Se debe seleccionar cuando se requieren recorridos bidireccionales o la eliminación arbitraria de nodos (**erase**) en medio de la estructura en tiempo $\mathcal{O}(1)$ (teniendo la referencia directa al nodo), o bien para operaciones de extracción en el extremo final (**popBack**) en $\mathcal{O}(1)$.
- ***DynamicArrayStack* y *DynamicArrayQueue*:** Son la elección preferente cuando el volumen de datos es predecible o cuando se requiere minimizar la fragmentación de memoria y aprovechar la localidad de referencia. La cola basada en arreglo circular previene la degradación por desplazamiento de índices, ofreciendo un rendimiento altamente eficiente.

5.2. Escenarios de Aplicación Real de Pilas y Colas

■ Casos de uso de Pilas (*LIFO*):

- **Gestión de la pila de llamadas (*Call Stack*):** Control de la ejecución de métodos, alcance de variables locales y llamadas recursivas en lenguajes de programación.
- **Mecanismos de Deshacer/Rehacer (*Undo/Redo*):** Procesamiento de estados anteriores en editores de texto y software de diseño gráfico.

■ Casos de uso de Colas (*FIFO*):

- **Planificación de tareas (*CPU Scheduling*):** Algoritmos de ordenamiento de procesos en el sistema operativo (p. ej., *Round Robin*).
- **Recorridos en Grafos:** Algoritmo de búsqueda en anchura (*BFS*) e implementación de ordenamientos topológicos (algoritmo de Kahn) para la resolución de dependencias.

5.3. Arreglos Dinámicos vs. Listas Enlazadas en Java

Los arreglos dinámicos se destacan principalmente por su bajo consumo de memoria auxiliar. Dado que almacenan los datos de forma contigua en memoria permiten un acceso aleatorio indexado en tiempo constante $\mathcal{O}(1)$ sin requerir almacenamiento adicional por elemento. Sin embargo, su principal desventaja radica en la latencia por redimensionamiento: cuando la capacidad del arreglo se agota, la operación de inserción cae a un peor caso de $\mathcal{O}(n)$ debido a la reasignación de memoria y al copiado completo de los elementos, además de poder generar cierta memoria inutilizada debido al factor de crecimiento.

Por otro lado, las listas enlazadas ofrecen como ventaja principal una asignación de memoria dinámicamente estricta y un rendimiento predecible, garantizando que las inserciones y eliminaciones en los extremos se ejecuten en tiempo $\mathcal{O}(1)$ sin sufrir jamás los picos de redimensionamiento. Su mayor desventaja en la Máquina Virtual de Java (JVM) se encuentra en la alta sobrecarga de memoria y la constante presión sobre el Recolector de Basura (*Garbage Collector*) debido a la rápida asignación y liberación de objetos dinámicos.

Anexo 1

Cuadro 4: Tiempos de ejecución acumulados en microsegundos para *SinglyLinkedList*

(N)	addAfter	addBefore	erase	find	getSize	isEmpty	popBack	popFront	pushBack	pushFront
10	0	0	0	0	0	0	0	0	0	198,12
100	198,38	0	606,34	0	0	0	0	0	0	0
1000	1191,86	1785,58	4865,86	2106,16	0	0	1907,46	205,78	1198,98	0
10000	79296,66	213838,92	707527,56	190888,48	201,02	0	408650,92	0	80684,82	0
100000	11224738,88	23328046,46	177669376,50	22223775,00	199,74	397,6	182555531,50	1864,44	10885833,46	0

Cuadro 5: Tiempos de ejecución acumulados en microsegundos para *SinglyLinkedListTail*

(N)	addAfter	addBefore	erase	find	getSize	isEmpty	popBack	popFront	pushBack	pushFront
10	0	0	0	0	0	0	0	0	0	0
100	0	0	604,58	0	0	0	0	196,18	0	198,60
1000	1202,40	1800,46	4260,52	2029,00	0	0	1672,56	201,76	0	0
10000	74065,18	199575,74	1007083,72	220715,56	0	0	613371,48	0	0	304,34
100000	12179901,54	24430254,82	215502664,10	19816329,36	0	399,28	201797907,90	996,86	2440,22	2744,80

Cuadro 6: Tiempos de ejecución acumulados en microsegundos para *DoublyLinkedList*

(N)	addAfter	addBefore	erase	find	getSize	isEmpty	popBack	popFront	pushBack	pushFront
10	0	0	0	0	0	0	0	0	0	401,74
100	0	201,78	202,74	401,30	0	0	0	0	0	0
1000	1108,28	1782,68	4380,80	2012,54	0	0	1607,18	0	1074,52	0
10000	79185,38	213637,04	680926,40	185912,46	0	0	400135,52	198,52	82722,22	605,96
100000	13420073,06	27041040,30	180415732,70	23542248,60	302,96	253,4	333001923,60	1406,00	11993612,32	2644,84

Cuadro 7: Tiempos de ejecución acumulados en microsegundos para *DoublyLinkedListTail*

(N)	addAfter	addBefore	erase	find	getSize	isEmpty	popBack	popFront	pushBack	pushFront
10	0	0	0	0	0	0	0	0	0	0
100	396,90	0	796,86	0	0	0	0	0	0	0
1000	1179,30	2217,48	4399,88	1905,54	0	0	0	0	0	0
10000	72807,56	181413,70	679629,88	161699,52	0	0	200,82	402,28	203,24	0
100000	12720363,56	27044629,18	168086631,00	22484616,38	0	479,18	2608,62	4954,86	3173,26	7146,72

Cuadro 8: Tiempos de ejecución acumulados en microsegundos para *DynamicArrayStack*

(N)	delete	getSize	isEmpty	peek	pop	push
10	0	0	0	0	0	0
100	404,38	0	0	0	0	0
1000	2916,20	0	0	0	0	0
10000	158172,46	0	0	200,76	0	0
100000	28024617,44	460,1	661,76	1886,62	0	1696,52

Cuadro 9: Tiempos de ejecución acumulados en microsegundos para *DynamicArrayQueue*

(N)	delete	dequeue	enqueue	front	getSize	isEmpty
10	0	0	0	0	0	0
100	0	197,64	0	0	0	102,62
1000	3833,96	0	0	0	0	200,42
10000	236116,76	0	505,32	0	0	0
100000	41208081,14	903,9	6112,2	198,92	202,54	1255,92